

Laboratorio 2 - Parser CSV strutturato

Obiettivo

Evolvere il programma del Laboratorio 1 in modo che il CSV non venga più trattato come testo grezzo, ma venga **interpretato** in una rappresentazione strutturata e tipizzata.

Il programma deve:

1. Leggere un file CSV
2. Riconoscere l'header (prima riga) come nomi di colonna
3. Suddividere ogni riga successiva in campi tipizzati (intero, decimale, testo)
4. Validare la coerenza dei tipi tra le righe e con l'intestazione
5. Stampare il contenuto in modo leggibile

Il laboratorio serve a consolidare:

- `enum` e `struct` per modellare dati complessi
- `Result` e propagazione/gestione degli errori
- `impl` di metodi e trait standard (`Display`)
- uso di crate esterne (`clap`, `anyhow`)

Specifiche funzionali

Interfaccia

Il programma deve essere eseguibile come nel laboratorio precedente:

```
cargo run -- <file.csv>
```

Rappresentazione dei dati

Il CSV non deve più essere trattato come testo grezzo. Ogni campo deve essere interpretato e rappresentato con il tipo appropriato: intero, decimale o testo. Come modellare questa distinzione e come organizzare i dati analizzati è a vostra scelta.

Il risultato del parsing deve implementare il trait `Display` per poter essere stampato in modo leggibile.

Parsing

La prima riga del file è l'**header**: contiene i nomi delle colonne, separati da virgola.

Ogni riga successiva contiene i dati. Il parser deve:

1. Verificare che il numero di campi corrisponda al numero di colonne nell'header
2. Interpretare ogni campo come intero, decimale o testo
3. Verificare che tutte le colonne sia tra loro di tipo omogeneo

Se una riga ha un tipo diverso da quello atteso per una colonna, è un errore.

Esempio con header `iniziale,età,voto`:

```
A,25,8.5    ← OK (Testo, Intero, Decimale) - stabilisce i tipi
B,30,9.0    ← OK (Testo, Intero, Decimale)
C,abc,7.5   ← ERRORE: colonna 2 (età) atteso Intero, trovato Testo
```

Output

Al termine dell'esecuzione, se non sono stati trovati errori, il contenuto del CSV deve essere stampato in modo leggibile. Il formato è libero, ma deve essere chiaro e coerente. Un esempio possibile è una tabella allineata come la seguente:

```
nome | età | voto
-----+-----+-----
Alice |  25 |   8.5
Bob   |  30 |   9.0
```

Gestione errori (requisito obbligatorio)

Il programma **non deve andare in panic** in uso normale.

Devono essere gestiti:

- File inesistente o non leggibile
- File vuoto (nessuna riga)
- Numero errato di campi in una riga
- Tipo di campo incoerente con la prima riga di dati
- Errori di I/O durante la lettura

In caso di errore:

- Collezionare l'errore in una struttura dati opportuna e, se possibile, continuare con la lettura del file
- Se è stato rilevato almeno un errore, terminare con exit code diverso da 0 e riportare la descrizione di tutti gli errori raccolti

I singoli messaggi di errore devono contenere abbastanza contesto da essere utili (es. numero di riga, nome della colonna, tipo atteso vs trovato).

Crate consigliate

- **clap** - per la gestione degli argomenti da riga di comando (sostituisce la gestione manuale di `std::env::args` del Lab 1)
- **anyhow** - per la gestione degli errori, semplifica la propagazione con `Result<T, anyhow::Error>` e la macro `anyhow!()` per creare errori con messaggi descrittivi

È vietato usare `unwrap()` o `expect()` su input proveniente dall'utente o dal filesystem.

Struttura del progetto

Struttura minima consigliata:

```
src/  
├── main.rs  
├── cli.rs  
└── csv.rs
```

`main.rs`

- Orchestrazione
- Invoca parsing argomenti
- Invoca la lettura e il parsing del file CSV
- Gestisce gli errori e stampa il risultato

`cli.rs`

- Parsing argomenti con `clap`

`csv.rs`

- Tipi e logica per rappresentare e parsare il CSV
- Implementazione di `Display`

Questo modulo non deve terminare il processo né stampare su `stderr`: deve limitarsi a restituire `Result`.

Criteri di autovalutazione

40% - Correttezza funzionale

- Parsing dell'header corretto
- Campi tipizzati correttamente (intero, decimale o testo)
- Validazione dei tipi coerente tra le righe
- Output leggibile

30% - Robustezza

- Nessun panic
- Errori gestiti correttamente con messaggi utili
- Exit code appropriato

20% - Struttura e chiarezza

- Separazione moduli
- Codice leggibile
- Uso idiomatico di `enum`, `struct`, `Result`

10% - Processo di sviluppo

- Repository Git con messaggi significativi, visibili con `git log`
- Deve compilare eseguendo `cargo build`
- Deve funzionare eseguendo `cargo run -- <file.csv>`

Obiettivo didattico

Alla fine di questo laboratorio dovrete essere in grado di:

- Definire `enum` con varianti che contengono dati diversi
- Definire `struct` con campi complessi e implementarne i metodi
- Implementare il trait `Display` per la stampa personalizzata
- Usare `Result` e `anyhow` per propagare errori in modo pulito
- Usare crate esterne (`clap`, `anyhow`) in un progetto Cargo
- Separare la logica di dominio dall'I/O e dalla CLI